

HW4 - Neural Networks

Vanja Stojanović

May 2026

1 ANNClassification

A multi-layer fully-connected ANN with analytically derived gradients. The unit is `ANNLayer` (weight matrix W , bias b , pluggable activation pair (f, f')). Forward caches $A_{prev}, Z = A_{prev}W + b, A = f(Z)$; backward computes $dZ = dA \odot f'(A, Z)$, $dW = A_{prev}^\top dZ$, $db = \sum dZ$, and pushes $dA_{prev} = dZW^\top$ upstream. Hidden layers use sigmoid. The output layer is softmax, but the gradient enters through `backward_from_dZ` with $dZ = (P - Y_{onehot})/n$: softmax + cross-entropy collapses to the canonical-link residual, the same identity as $\hat{y} - y$ (OLS) and $p - y$ (logistic). Optimization is full-batch gradient descent.

1.1 Smallest networks for doughnut and squares

For each candidate architecture we trained 5 random initializations for 5000 epochs at $lr = 0.5$ and counted seeds that reached perfect training accuracy (Tables 1, 2).

Table 1: doughnut.tab; reliable success at 5/5 seeds.

units	best acc	mean acc	success	params
[]	0.384	0.384	0/5	6
[2]	0.903	0.902	0/5	12
[3]	1.000	1.000	5/5	17
[4, 2]	1.000	1.000	5/5	28

Table 2: squares.tab; reliable success at 5/5 seeds.

units	best acc	mean acc	success	params
[]	0.628	0.628	0/5	6
[4]	1.000	0.994	3/5	22
[10]	1.000	1.000	5/5	52
[4, 2]	1.000	1.000	5/5	28

Doughnut: [3] (17 parameters) suffices. Three sigmoid half-planes are enough for the output layer to compose into a closed circular-ish boundary; [2] caps at 90%. **Squares:** [4, 2] (28 parameters) is the smallest reliable architecture, beating single-layer [10] (52). The extra layer lets the network compose features for disjoint rectangle regions, which a single hidden layer struggles to capture from random init. **No-hidden** ([]) is multinomial logistic regression and cannot represent either non-linear boundary; both stay below 70%.

1.2 Why standardization and gradient $/n$

Both are needed for a single lr to converge across datasets of different scale and size. Doughnut, [8], $lr = 0.5$, 3 seeds (Table 3).

Table 3: Ablation: standardization (std) and division of gradient by n ($/n$).

config	mean acc	final loss	failure mode
std=ON, $/n$ =ON	1.000	0.004	(baseline)
std=OFF, $/n$ =ON	1.000	0.021	sigmoid saturates on raw inputs
std=ON, $/n$ =OFF	0.667	4.59	step $\sim n \times$ too large; oscillates
std=OFF, $/n$ =OFF	0.477	13.17	both compounded

Standardization ($\tilde{X} = (X - \mu)/\sigma$) keeps first-layer pre-activations in sigmoid’s non-saturated range; without it, $|Z|$ grows with input scale, the derivative $\sigma(z)(1 - \sigma(z))$ collapses, and gradients vanish. **Gradient $/n$** matches the mean-loss convention: without it, dW sums contributions from all n samples, so an lr tuned for $n = 4$ is roughly n -times too aggressive at $n = 216$. The two fixes target different points in the pipeline (forward saturation vs. update step size) and their failure modes are independent.

1.3 Learning-rate tuning per dataset

We swept $lr \in [0.01, 5.0]$ on each dataset’s smallest reliable architecture, 5000 epochs, seed 0, and report the first epoch where the loss drops below 0.01 as a proxy for convergence speed (Table 4).

Table 4: Learning-rate sweep on the smallest reliable architecture per dataset.

lr		0.01	0.05	0.1	0.5	1.0	2.0	5.0
doughnut, [3]	final loss	0.645	0.082	0.034	0.005	0.003	0.001	0.0006
	acc	0.773	1.000	1.000	1.000	1.000	1.000	1.000
	epoch loss<0.01	n/a	n/a	n/a	2761	1381	690	273
squares, [4, 2]	final loss	0.693	0.688	0.423	0.005	0.002	0.001	0.0004
	acc	0.515	0.705	0.873	1.000	1.000	1.000	1.000
	epoch loss<0.01	n/a	n/a	n/a	3328	1667	904	1720

Doughnut tolerates a wide lr range. Anything in $[0.05, 5.0]$ reaches 100% accuracy. Higher lr is monotonically faster: $lr = 5.0$ hits loss < 0.01 in 273 epochs, ten times faster than $lr = 0.5$, with no upper-end instability.

Squares is more demanding. $lr \in [0.01, 0.1]$ does not even reach 100% accuracy in 5000 epochs, and the convergence-speed curve is non-monotonic: $lr = 2.0$ is fastest (904 epochs), while $lr = 5.0$ is almost twice as slow because the higher step starts overshooting.

This is consistent with the loss-landscape difference between the two problems. Doughnut has a simple closed boundary; the loss surface is well-conditioned and large steps remain in productive directions. Squares requires composing several disjoint regions; the surface has narrower valleys, so step sizes that suit doughnut can overshoot here. **Per-dataset recommendation:** $lr = 2.0$ is the best single choice that works for both; if tuning per dataset, doughnut prefers $lr = 5.0$ and squares $lr = 2.0$. We keep $lr = 0.5$ as the conservative library default and surface `epochs` and `lr` as instance attributes for callers to override.

2 PyTorch clone

We re-implement the same architecture in PyTorch (`ANNClassificationTorch`, same file). Both clones share: sigmoid hidden layers, linear output with softmax folded into the loss, plain SGD, full-batch updates, mean cross-entropy, identical Xavier-style init $\mathcal{N}(0, 1/\sqrt{d_{in}})$ seeded from numpy, and the same input standardization. The difference is that PyTorch builds an autograd graph and computes gradients automatically, whereas ours derives them analytically and stores per-layer dW, db .

Table 5: Side-by-side, matched seeds and hyperparameters, 2000 epochs.

dataset	impl	time (s)	final loss	acc	agreement
doughnut, [3], $lr = 0.5$	ours	0.069	0.0146	1.000	$\max \Delta P = 4 \cdot 10^{-7}$
	pytorch	1.043	0.0146	1.000	
squares, [4, 2], $lr = 1.0$	ours	0.125	0.0072	1.000	$\max \Delta P = 9 \cdot 10^{-7}$
	pytorch	0.439	0.0072	1.000	

Numerical agreement. Final loss matches to 4 decimals on both datasets. Per-sample probability differences peak at $\sim 10^{-6}$ and the loss curves agree pointwise to $\sim 10^{-7}$. The remaining gap is dominated by float32 (PyTorch’s default) versus float64 (ours), plus accumulator-order differences inside torch’s matmul. With matched init and matched optimizer, the two trajectories are otherwise the same algorithm.

Speed. Ours is faster on these tiny networks ($\approx 15\times$ on doughnut, $\approx 3.5\times$ on squares). The reason is per-operation Python overhead: PyTorch builds an autograd graph and dispatches to its kernels for every matmul and activation, which dominates when each call only operates on a few hundred floats. Our numpy code does the same handful of matrix multiplies with no graph bookkeeping. The advantage would invert at much larger sizes where the matmuls themselves dominate (and even more so on GPU, which we did not exercise).

Implementation cost. The PyTorch version is shorter: no `backward`, no `backward_from_dZ` shortcut, no manual chain-rule bookkeeping. The model is built by stacking `nn.Linear` and `nn.Sigmoid` into `nn.Sequential`; `loss.backward()` fills every parameter’s `.grad`, and `optimizer.step()` applies the update. Cross-entropy and softmax are merged in `nn.CrossEntropyLoss`, which gives the same numerically stable and analytically clean $\partial L/\partial Z = (P - Y_{onehot})/n$ we derived by hand. In practice that means: the things we had to think carefully about (the softmax+CE combine, the $/n$ normalization, the saturation-aware init) are exactly what the framework already does.

Conclusion. The two implementations are equivalent in result but trade speed for code clarity in opposite directions: ours is faster but requires the analytical derivation; PyTorch is slower for small models but offloads the math to a generic engine. For an assignment that asks for analytically derived gradients, the numpy version is the right vehicle and the agreement to 10^{-6} is independent confirmation that our backward pass is correct.

2.1 Extensions

We extend the numpy implementation with three orthogonal features. All share the same `ANNLayer` primitive and the same training loop; the diff per feature is small.

Regression (ANNRegression). The output layer is a single `LINEAR` unit, the loss is mean squared error, and $\partial L/\partial Z_{out} = 2(\hat{y} - y)/n$, the Gaussian-target counterpart of the categorical residual $(P - Y_{onehot})/n$ (same canonical-link GLM identity). The forward pass, the per-layer

backward, the standardization, and the optimizer are identical to classification. `predict` squeezes the trailing axis to $(n,)$. All five `ANNRegression` test cases pass without further tuning; the same $lr = 0.5$, `epochs = 5000` defaults work.

Per-layer activations. The constructor accepts `activations`: a list of strings ("`sigmoid`", "`relu`", "`linear`") or (f, f') tuples, one per hidden layer. The output activation is task-determined (softmax for classification, linear for regression) and not user-overridable. ReLU on `units=[3]` on doughnut beats sigmoid by $4\times$ in convergence speed: ReLU reaches loss < 0.05 at epoch 173 vs. epoch 728 for sigmoid, with final loss roughly $4\times$ lower as well. ReLU's gradient is exactly 1 in the active region, so it doesn't suffer the saturation flattening that bounds sigmoid's effective gradient at ≤ 0.25 .

L2 regularization. The weight update becomes $W \leftarrow W - lr \cdot (dW + \lambda W)$; biases are exempt ($b \leftarrow b - lr \cdot db$). Sweep on doughnut, `units=[10]`, 5000 epochs:

Table 6: Effect of λ on weight magnitude and fit (doughnut, [10]).

λ	final loss	acc	$\ W\ $	$\ b\ $	note
0.000	0.004	1.000	26.60	12.27	unconstrained, perfect fit
0.001	0.030	1.000	17.59	11.79	$\ W\ $ -34%, fit preserved
0.010	0.383	0.898	7.97	5.11	accuracy starts to drop
0.100	0.692	0.523	0.00	0.18	W collapses to zero
1.000	0.692	0.523	0.00	0.07	same regularizer-only fixed point

$\|W\|$ shrinks monotonically with λ as expected. $\|b\|$ also drops, but less and indirectly: biases get no L2 penalty, the shrinkage on biases is just the gradient-flow side-effect of weights pulling activations smaller. At $\lambda \geq 0.1$ the regularizer dominates and W collapses to zero, leaving softmax to default to its uniform output (accuracy $\approx 1/K$).